

计算机体系结构

新型计算机研究所

陈 衡

西一楼 A 段415室

Email: hengchen@xjtu.edu.cn

第2次上机实验

◆ 流水线性能分析II

- 实验目的
- 实验平台
- 实验内容和步骤
- 实验结果

流水线性能分析

◆ 实验目的

- 加深对计算机流水线基本概念的理解；
- 理解MIPS 结构如何用5 段流水线来实现，理解各段的功能和基本操作；
- 加深对数据冲突、结构冲突、控制冲突的理解，并能够分析这些冲突对CPU 性能的影响；
- 进一步理解解决数据冲突的方法，掌握如何应用定性技术来减少数据冲突引起的流水线停顿。

流水线性能分析

◆ 实验平台

- (1) 自己设计一个流水线模拟器(简称:模拟器A)
- (2) 使用开源流水线模拟器(简称:模拟器B);

流水线性能分析

◆ 实验步骤，至少实现以下功能

- 能够模拟MIPS的5段流水线，以课堂讲授为参考；
- 熟悉图形交互或者命令交互；
- 熟悉单步执行（一次一个时钟周期）、执行到断点、执行到程序结束；
- 掌握流水线各个段、寄存器状态的查看；
- 掌握是否使用定向路径的功能选项；
- 提供程序执行后的性能统计分析；
- 按照MIPS 语法，至少支持 load、store、add、beqz 操作；
- 要执行的程序可以直接输入，或通过文件载入的方式运行
- 可以使用你喜欢的**任何语言**来实现；
- 可参考上机实验 I 中的两个模拟器中实现的功能

流水线性能分析

- ◆ 实验步骤，至少实现以下场景
 - 没有任何冲突的流水线场景；
 - 有至少一次的RAW 冲突；
 - 有至少一次的分支跳转；
 - 使用课堂讲授的内容，对上述每一种场景进行分析

流水线性能分析

◆ 场景举例

- 没有任何冲突的流水线场景；

```
.text
main:
ADDIU $r1,$r0,A
LW $r2,0($r1)
LW $r3,4($r1)
ADDIU $r4,$r0,B
SW $r2,0($r4)
SW $r2,4($r4)
.data
A: .word 1, 2, 4, 8
B: .word 1, 2, 3, 4
```

流水线性能分析

- ◆ 实验结果，
- ◆ (1) 模拟器A和模拟器B需提交完整的源代码、编译过程
- ◆ (2) 完整的实验报告至少包含：
 - 模拟器A、B的设计思想、特色；
 - 测试代码组合在模拟器A或B中的执行情况；
 - 测试代码在模拟器的执行过程，以图文并茂的方式呈现；
 - 实验感悟

流水线性能分析

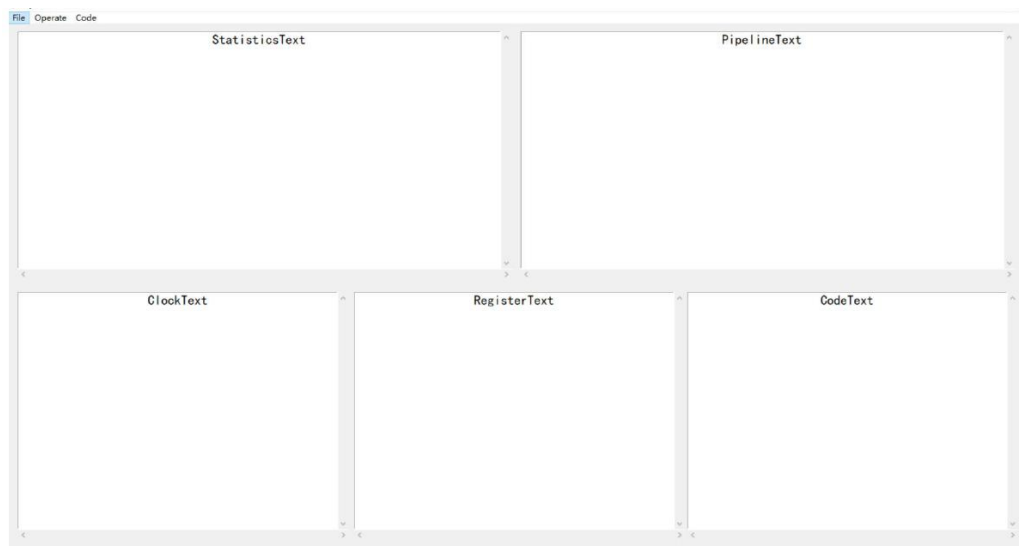
◆ 实验结果，举例

四、实验代码

实验代码主要依赖于 Tkinter 图形化库搭建，此外引用 typing 库控制函数输出类型，引用 re 库进行字符串解析。

相关代码细节均为我独立完成，未参考流水线模拟器任何开源代码，但模拟器 A 的搭建界面与应实现的功能均以 WinDLX 模拟器为基础，具体功能包括但不限于图形交互、启用调试模式、重置 DLX、重置所有数据、退出程序、单步执行、断点执行、执行到程序结束、流水线各个段与寄存器状态的查看、提供定向路径的功能选项、提供程序执行后的性能统计分析、通过文件载入的方式运行程序、支持‘LW’，‘SW’，‘ADD’，‘SUBI’，‘BEQZ’，‘J’操作；

为方便老师查验，本代码已上传至 [github](#) 且未告知他人，链接如下：



```
StatisticsText
Total:
13 Cycle(s) executed.
ID executed by 6 Instruction(s)
0 Instruction(s) currently in Pipeline.

Hardware configuration:
faddEX-Stages: 1, required Cycles: 6.
fmulEX-Stages: 1, required Cycles: 7.
fdivEX-Stages: 1, required Cycles: 10.
Forwarding enabled.

Stalls:
RAW stalls: 2 (15.38% of all Cycles).
WAW stalls: 0 (0.00% of all Cycles).
Structural stalls: 0 (0.00% of all Cycles).
Total stalls: 2 (15.38% of all Cycles).

Conditional Branches:
Total: 0 (0.00% of all Cycles), thereof:
    taken: 0 (0.00% of all Conditional Branches).
    not taken: 0 (0.00% of all Conditional Branches).

Load/Store Instructions:
Total: 4 (30.77% of all Cycles), thereof:
    Loads: 2 (50.00% of all Load/Store Instructions).
    Stores: 2 (50.00% of all Load/Store Instructions).
```

流水线性能分析

4. 实验结果

◆ 实验结果，举例

(1) 模拟器程序设计。

由于模拟器源代码过长(648 行, 12K), 因此不在本文中展示, 可在附件 NaiveSim.cpp 中查看。

由于使用了头文件 windows.h, 该程序仅支持 Windows 系统。

编译命令: g++ Naivesim.cpp -o Naivesim.exe

模拟器设计思路:

在命令行中使用键盘按键以及标准输入进行交互, 使用简单的清屏、颜色设置等操作实现有一定特色且方便理解的命令行交互界面。

5. 实验感悟

(1) 尽管之前在组成原理课上自己实现过 MIPS 流水线 CPU, 但是在实现 MIPS

流水线模拟器时, 花费的时间还是超出了我的预期。我认为一方面这是因为组成原理和体系结构的视角和层次不同, 需要关注的东西和需要决定的细节不一样; 另一方面是也需要自己去思考和实现演示模拟结果的接口, 这是我之前没有做过和思考过的。因此, 我对于体系结构所要解决的任务和其视角层次有了更深刻的理解。

(2) 同时, 在实验中也实现了定向技术的开关功能, 支持用定向技术减少或

消除 RAW 冲突。在实验过程中可以看到, 定向技术的效果非常显著, 能直接将半数的停顿降低到 6% 的程度, 除了相邻的 LW-ADD 冲突外其他冲突均可消除。从这里我也更加感受到体系结构上优化的巨大威力。

定向已关闭

```
0000: ADDIU $r 8, $r 0, 0
0001: ADDIU $r 9, $r 0, 1
0002: ADDIU $r10, $r 0, 2
0003: LW    $r 8, $r 8, 0
0004: LW    $r 9, $r 9, 0
0005: LW    $r10, $r10, 0
0006: ADD    $r11, $r 0, $r 8
0007: ADD    $r12, $r 8, $r 9
```

```
$r00=0, $r01=0, $r02=0, $r03=0, $r04=0, $r05=0, $r06=0, $r07=0
$r08=0, $r09=1, $r10=0, $r11=0, $r12=0, $r13=0, $r14=0, $r15=0
$r16=0, $r17=0, $r18=0, $r19=0, $r20=0, $r21=0, $r22=0, $r23=0
$r24=0, $r25=0, $r26=0, $r27=0, $r28=0, $r29=0, $r30=0, $r31=0
```

```
* 按 1 单步执行
* 按 2 设置断点
* 按 3 连续执行
* 按 A 开启定向
* 按 S 显示统计
* 按 D 显示内存
* 按 Z 文件导入
* 按 X 手动输入
* 按 Esc 退出
```

作业提交要求

- ◆ 使用思源学堂
 - <http://lms.xjtu.edu.cn/>
- ◆ 作业提交的截至时间为2周(5月30日)，以思源学堂的时间为准
- ◆ 格式要求：pdf
- ◆ 逾期提交会做扣分处理